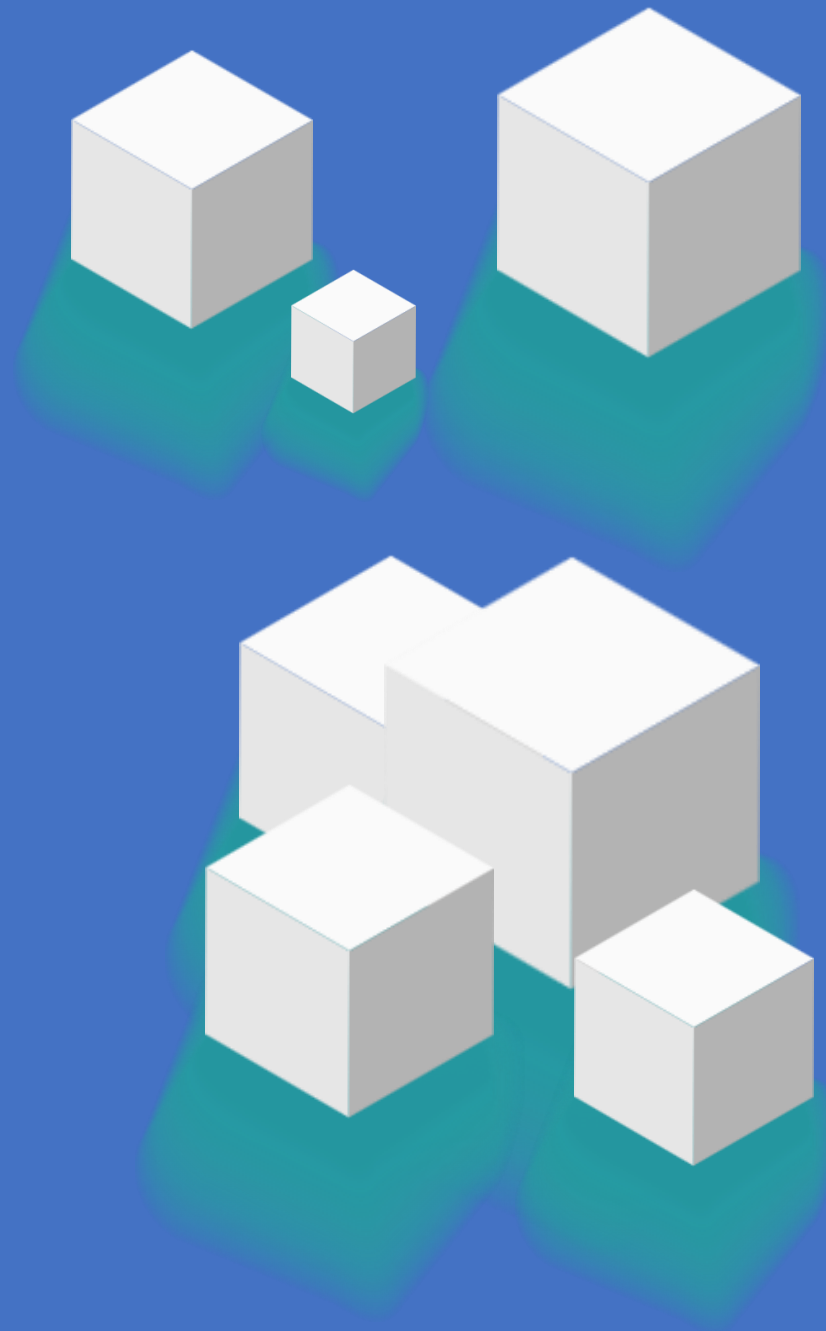
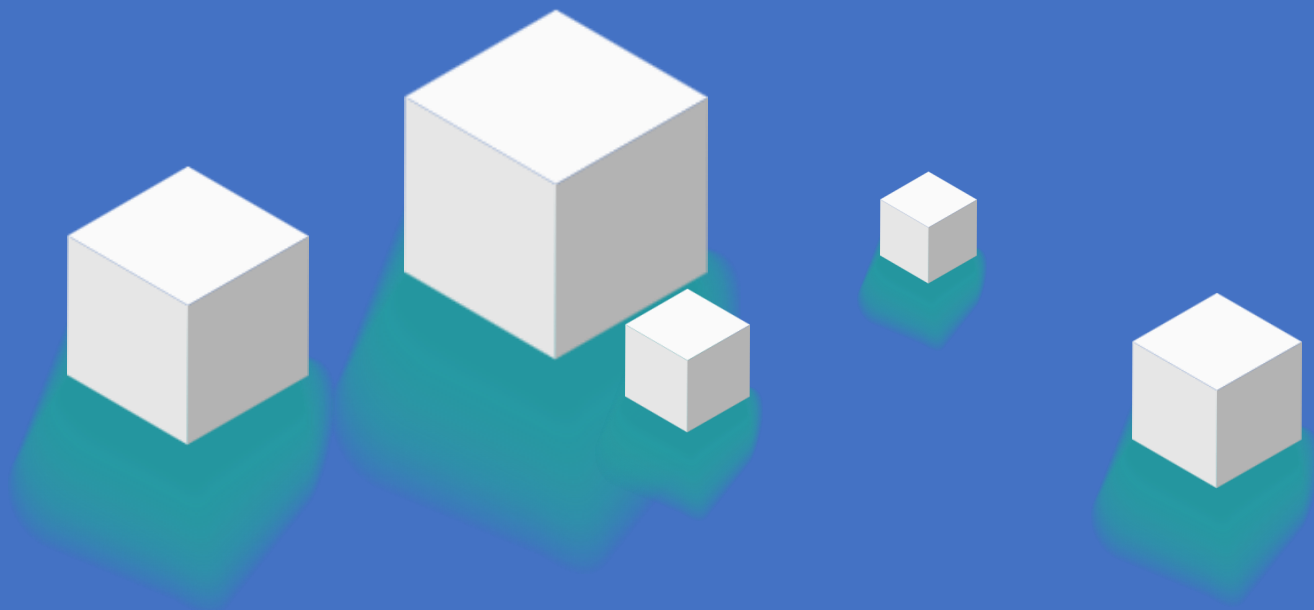


智能研报生成



>> 今天的学习目标

智能研报生成

- 国泰君安五步法

信息差、逻辑差、预期差、催化剂、结论

CASE：海立股份（光刻机概念）

CASE：易鑫集团（汽车金融）

CASE：尿素期货（大宗商品）

- 搭建AI研报

AI模式选择

研报数据预处理

DeepAgents框架

Skills 系统

SkillsMiddleware 工作原理

两种架构模式对比

自定义 Skill

CASE：content-builder-agent（内容生成 Agent）

CASE：deep_research（深度研究 Agent）

CASE：text-to-sql-agent（SQL 查询 Agent）

中间件栈与执行流程

- CASE：撰写AI投研报告

国泰君安五步法

国泰君安五步法

Thinking: 什么是五步法?

国泰君安证券研究所使用的投资分析方法论。从信息到逻辑到结论的完整推理链。

=> 找到市场 还没意识到的 投资机会。

Step 1: 信息差 → Step 2: 逻辑差 → Step 3: 预期差 → Step 4: 催化剂 → Step 5: 结论

五步法是国泰君安证券研究所所长提出的方法论。

核心是一套认知博弈框架，而非单纯的基本面分析。

=> 目标是找到市场还没意识到的投资机会。

=> 做证券投资很重要的是要去观察众人内心世界的变动，而不能执着于自己的一些想法...投资的目的是参悟智慧，钱只是一个心灵的符号。

国泰君安五步法

Step 1: 信息差

核心问题：市场还不知道/忽视了什么关键数据？

寻找目标

- 隐藏亮点：附注中的关联交易变化、非经常性损益中的规律性收入、新业务增长信号。
- 隐藏风险：应收账款/存货异常变化、现金流与利润背离、资产减值潜在风险。
- 关键数据：3-5个最能体现信息差的核心数据点。

国泰君安五步法

Thinking: 为什么从信息差开始?

因为市场不是全知的。财报有200+页，大部分投资者只看摘要和利润表。

附注、现金流量表、关联交易中藏着大量被忽视的信息 => AI可以 通过 RAG 可以全文检索，找到人类容易遗漏的细节。

比如：某公司利润表显示净利润增长10%（看似平淡），但附注显示本期计提了一笔2亿元的商誉减值。如果没有减值，实际净利润增长率约25% => 这就是信息差。

国泰君安五步法

Step 2: 逻辑差

核心问题：别人看到了数据，但推理错在哪里？

分析维度

- 常见误读：市场对该公司最常见的错误认知。
- 因果重构：重新构建正确的因果逻辑链。
- 预判修正：市场错误预判 vs 正确预判。

比如：某消费品公司毛利率下降了2个百分点。

- 市场解读：竞争加剧，产品力下降 -> 看空。
- 正确逻辑：公司主动降价抢市场份额，虽然毛利率降了，但销量增长30%，净利润反而增长

=> 这就是逻辑差。

国泰君安五步法

Step 3: 预期差

核心问题：市场一致预期 vs 实际数据偏离多大？

指标	市场一致预期	财报实际/合理估计	偏离幅度
营收增速	15%	22%	+7pp（正向预期差）
净利润增速	12%	8%	-4pp（负向预期差）
毛利率	45%	47%	+2pp（正向预期差）

思考：预期差是一次性的还是可持续的？净预期差的方向和强度。

国泰君安五步法

Thinking: 没有市场一致预期数据怎么办?

可以参考：券商研报的预测均值（Wind、东方财富iFinD）、行业平均增速、公司历史增速趋势、管理层之前的业绩指引。

如果都没有，LLM可以基于行业经验给出合理估计并注明。

国泰君安五步法

Step 4: 催化剂

核心问题：什么事件会触发市场重新评估这家公司？

- 短期催化剂（1-3个月）

即将发布的财报/业绩预告；重大合同/订单公告；股东大会决议；增持/回购计划执行。

- 中期催化剂（3-12个月）

新产品/新业务放量；行业周期拐点信号；并购/资产重组落地；机构调研密集 -> 评级变化。

- 潜在负面催化剂

限售股解禁；大股东质押比例高；行业政策收紧。

国泰君安五步法

Thinking: 为什么催化剂如此重要?

好公司 + 好价格不一定马上涨, 需要一个引爆点让市场重新关注。

催化剂就是这个引爆点。没有催化剂的投资逻辑 = 可能正确但时机不对。

年报催化剂

Thinking：如何考虑年报这个时间点的催化剂？

A股年报披露截止时间是每年4月30日。所有上市公司必须在2026年1月1日至4月30日之间完成披露。

现在正是2026年4月初，年报季已进入最后冲刺阶段——越往后，披露的公司越多，且通常业绩较差的公司倾向于压线披露（即4月下旬）。

那都需要关注哪些？

年报催化剂

买预期，卖现实 (A股年报季的经典博弈模式)

买预期阶段 (去年11月-今年3月)

- 市场炒作的是业绩预增的想象力和估值切换的空间
- 资金基于行业景气度、订单情况、预告信息等提前布局
- 此时股价往往已充分反映甚至透支业绩利好

卖现实阶段 (4月前后)

- 当正式年报披露，确定性落地，故事讲完
- 聪明资金选择在业绩兑现时获利了结
- 即使业绩很好，也可能出现 利好出尽是利空 的下跌

年报催化剂

当前时间点（4月初）的逻辑

1) 时间窗口紧张，雷区临近

4月最后两周（尤其是4月20日后）是年报密集披露期，业绩变脸、非标审计意见、分红不及预期等风险集中暴露
=> 此时市场情绪通常趋于谨慎。

2) 分红与高送转的博弈

年报季的另一条暗线是分红预案：

- 高分红、高送转股票往往在预案公告前有资金埋伏
- 但监管对忽悠式高送转严格监管，需关注分红是否可持续、是否匹配业绩增速

3) 一季报的叠加效应

4月同时是2026年一季报的披露窗口（4月1日-4月30日）。市场更关注业绩连续性：

- 如果2025年年报好但2026Q1大幅下滑 → 见光死
- 年报一般但Q1超预期 → 可能逆势走强

年报催化剂

4) ST与退市风险敏感期

4月是ST股票和退市风险股最危险的时刻：

- 连续两年亏损、净资产为负、审计非标意见等触发条件明朗化
- 4月最后一周常有跌停潮，需警惕持仓股票的财务风险

5) 机构调仓的明牌时刻

年报披露后，机构的最新持仓数据（通过十大流通股东、基金季报）完全暴露：

- 可关注机构新进场但股价未大涨的股票（存在滞后博弈机会）
- 警惕机构大幅减持但股价仍高的标的

距离财报公布日越来越近，尚未披露年报的公司（尤其是前期炒作充分的）风险收益比正在急剧恶化。

可以考虑将持仓向已披露+业绩验证+Q1趋势向好的标的集中，而不是去赌最后的惊喜。

国泰君安五步法

Step 5: 结论 + 风险闭环

综合前四步分析，给出最终投资建议。

核心观点：一句话总结。投资逻辑：3-5个要点。

投资评级：强烈推荐 / 推荐 / 中性 / 回避。

关键假设与风险：以及投资者应持续跟踪的3-5个KPI。

风险闭环：必须揭示哪个环节出错会导致整个结论崩溃。

比如：如果行业政策突然收紧 / 技术路线被颠覆 / 核心客户流失，整套投资逻辑将不再成立。

=> 这是专业研究与普通推荐的关键区别。

CASE：海立股份（光刻机概念）

Thinking：2025年8月光刻机板块异动，海立股份午后直线涨停创历史新高。背后的逻辑是什么？

海立股份（600619.SH）是一家老牌上海国资背景的制造业企业，1992年上市，全球领先的空调压缩机供应商之一。

核心业务：形成了冷暖关联解决方案及核心零部件+汽车零部件双主业格局

- 压缩机业务（收入占比约73%）：制冷转子式压缩机、制冷电机、铸件，主要供应家用/商用空调市场，年产能约3,600万台
- 汽车零部件（收入占比约25%）：新能源车电动涡旋压缩机、汽车空调系统、热泵系统等，服务新能源汽车热管理需求



CASE：海立股份（光刻机概念）

按照五步法进行分析：

- 信息差：公司多次澄清与光刻机重组没关系。但市场通过供应链调研发现，其制冷设备在光刻机散热环节有实质性供货突破，这部分信息未被主流投资者充分理解。
- 逻辑差：散户看到澄清公告理解为利空；专业资金看到的是 不重组但业务实质受益 的辩证逻辑。
- 预期差：市场主流预期公司与光刻机无关联，实际已有技术验证通过并批量供货的预期未被定价。
- 催化剂：7月17日互动易确认 大功率电子管产品在光刻机领域通过客户验证并批量供货 成为引爆点。

=>信息差 + 预期差

CASE：海立股份（光刻机概念）

表面信息（散户视角）： 公司多次公告澄清与光刻机重组无关，且明确表示光刻机用冷却系统业务规模极小——近三年销售额占营业收入比例低于0.003%。

实质进展（专业资金视角）： 公司控股子公司海立特冷（持股70%）专注于工业设备冷却系统，其大功率电子管冷却设备在光刻机散热环节已通过客户验证并批量供货。虽然当前营收占比极低，但这是从"0到1"的供应链卡位，具备高技术壁垒和国产替代价值。

市场催化剂： 2025年7月17日互动易平台确认供货突破后，7月18日、8月6日、8月19日股价多次涨停，年内累计涨幅一度超过100%。

业绩表现

2025年上半年业绩大幅预增，归母净利润预计3050万-3600万元，同比增长626%-757%，主要受益于空调压缩机销量增长（上半年销售2,444万台，同比+35.9%）及汽车零部件业务减亏

CASE：易鑫集团（汽车金融）

Thinking：2025年4月关税动荡期间，如何梳理投资逻辑？

预期差层级	市场主流认知	未被认知的事实/逻辑
第一层	正常增长推算净利润12亿	股权激励目标暗示实际可达20亿（+67%）
第二层	一般金融撮合工具	SaaS服务+大模型对城商行/农商行的长尾效应未被定价
第三层	仅做国内业务	关税战后中欧/东南亚全面拥抱中国，新能源汽车出海对海外业务巨大帮助
第四层	传统业务估值	并购+汽车后市场+电池保险等新业务增长潜力

催化剂：5月一季报业绩披露将成为验证第一重预期差的关键节点。

=> 预期差可以分层叠加。当多重预期差被逐个验证时，股价会出现阶梯式上涨。



CASE: 珀莱雅 (美妆行业)

预期差：2022年珀莱雅净利润同比增长41.88%，远超分析师预期的7.68亿元。

逻辑差：多数投资者用传统化妆品公司渠道库存周期逻辑分析，正确逻辑应是新消费品牌用户留存+复购的互联网逻辑。

- 传统化妆品分析框架：关注渠道库存周期

铺货多少、经销商压货、渠道库存水位 => 本质是把品牌当 渠道驱动的销售公司，业绩随渠道周期波动，增长靠线下扩张。

- 珀莱雅的真实逻辑：已转型为用户驱动的消费品公司，适用互联网DTC（直接触达消费者）分析框架

核心指标是用户留存率、复购频次、LTV（用户终身价值）。其大单品策略（如双抗精华、红宝石精华）不是简单推SKU，而是通过产品矩阵建立用户生命周期管理：引流款获客→利润款复购→高端款提客单。



CASE：尿素期货（大宗商品）

信息差：农业技术革命重塑需求曲线

滴灌密植技术让玉米种植密度大幅提高（每亩植株数增加），导致单位面积尿素施用量从30-35kg增至40-55kg——全国玉米种植面积6亿亩，理论增量需求就是60-120万吨。这一结构性变化被期货市场忽视，分析师仍按旧农业模型估算需求。

预期差：三重支撑打破看空一致预期

市场主流预期下半年价格中枢下移（基于产能释放、淡季逻辑），但实际存在三重向上驱动：

- 追肥刚需：高密度种植模式下，玉米拔节-抽穗期追肥强度加大，刚需刚性且集中
- 出口爆发：法检（法定检验）政策放松使出口窗口打开，缓解国内过剩压力
- 承储托底：国储/商储阶段性入库形成价格护城河

催化剂：当出口法检政策从 严检限出 转向 快速通关，瞬间点燃多头情绪，技术图形与基本面共振，完成从一致看空到趋势反转的切换

CASE：撰写AI投研报告

AI模式选择

Thinking: 你会选择工作流，还是Agent模式，生成投研报告？

比如，用户输入：帮我撰写 贵州茅台 投研报告

方法1：工作流模式

将投研场景，转化为 SOP（标准作业程序）的数字化：

```
steps = [  
    lambda: 获取基础信息(ticker, 强制字段=["主营业务", "实控人", "市值"]), # 不填满不往下走  
    lambda: 抓取最新财报(报告期=最近季度, 必含科目=["营收", "扣非净利", "经营现金流"]), # 数据缺口自动告警  
    lambda: 同业对比(可比公司= predefined_list, 指标= uniform_metrics), # 防止 Agent 随意更换对标企业  
    lambda: 估值计算(方法论=DCF_or_PE, 参数来源=上一步数据), # 计算逻辑固化，可复核  
    lambda: 风险扫描(固定检查清单=["政策风险", "库存周期", "渠道库存", "批价波动"]) # 防止遗漏专业风险点  
]
```

工作流的核心价值在于固定流程，以及Checklist 强制化

AI模式选择

方法2: Agent模式

相比于工作流的固定流程, Agent的优势在于:

- 信息来源: web_search 联网为主 + 本地 RAG 为辅
- 搜索策略: Agent 自主规划, 发现线索后追加搜索
- 工具组合: 按需调用 N 个工具 (搜索/RAG/行情/财务指标/对比)
- 输出方式: 直接在对话中输出 Markdown 研报
- 迭代能力: 多轮搜索, 每轮基于上一轮发现决定下一步

如果你把投研分析的本质看做 研究, 那么可以采用不断迭代的过程。

即AI搜到一条线索, 会触发新的问题, 然后继续深挖。

CASE：研报数据预处理

CASE：研报数据预处理

TO DO：搭建数据预处理脚本

负责将 data/reports/ 中的 PDF 研报和财报 转换为 Agent 可直接查询的结构化知识库。

=> 构建 FAISS 向量索引，供 Agent 进行语义相似度检索（RAG）。

用户提问时，Agent 将问题转化为向量，在 FAISS 中找到语义最相近的文本片段（chunks），
然后把这些片段作为上下文交给 LLM 生成回答。



让 Agent 不必在每次对话时重新解析 PDF，而是预先处理好数据，实现秒级检索响应。

CASE：研报数据预处理

搭建 preprocess.py, 最终产出:

- FAISS 向量索引

位置: data/vector_store/

- SQLite 文档元数据

位置: data/documents.db

- JSON chunk 缓存

位置: data/parsed/

使用方式:

```
# 增量处理新 PDF, 重建索引
```

```
python preprocess.py
```

```
# 清除旧记录, 全部重新处理
```

```
python preprocess.py --rebuild
```

```
# 查看已处理文档列表
```

```
python preprocess.py --list
```

```
# 只解析 PDF, 不构建 FAISS 索引 (节省 API 调用)
```

```
python preprocess.py --skip-index
```

CASE：研报数据预处理

- 目录结构

```
preprocess.py          # 本脚本
data/
reports/               # 输入: 原始 PDF 文件
    【上海证券】中芯国际深度研究报告.pdf
    财报_贵州茅台：贵州茅台2025年第三季度报告.pdf
    ...
parsed/                # 中间产物: 解析后的纯文本 + 页码 JSON
    *_full.txt          # 全文文本
    *_pages.json        # 逐页信息（含字符数）
vector_store/          # 输出: FAISS 索引
    index.faiss         # 向量索引文件
    index.pkl           # docstore（元数据）
    chunks.json        # 全部 chunk 文本+元数据（供 BM25）
    index_meta.json     # 索引统计信息
documents.db           # SQLite 文档元数据库
```

CASE：研报数据预处理

- 股票名称-代码映射表

基本内置了一个名称到代码的字典，用于从文件名和正文中快速识别股票。

这是元数据提取的基础。

```
STOCK_CODE_MAP = {  
    "中芯国际": "688981",  
    "贵州茅台": "600519",  
    "比亚迪": "002594",  
    "宁德时代": "300750",  
    "五粮液": "000858",  
    # ... 可按需扩展  
}
```

Step1：扫描与增量检测

Step1：扫描与增量检测

首先扫描 data/reports/ 中所有 .pdf 文件，然后与 SQLite 数据库中的已处理记录进行比对，只处理新增文件。

```
def main():  
    # 1. 初始化 SQLite  
    conn = init_db(db_path)  
  
    # 2. 扫描目录中的所有 PDF  
    pdf_files = [f for f in os.listdir(reports_dir) if f.endswith(".pdf")]  
  
    # 3. 增量检测: 哪些文件已经处理过?  
    processed = get_processed_files(conn) # SELECT filename FROM  
documents  
    new_files = [f for f in pdf_files if f not in processed]
```

```
# 4. 检查统一索引是否存在  
index_exists = os.path.exists("data/vector_store/index.faiss")  
  
# 关键逻辑: 三种情况  
if not new_files and index_exists:  
    # 没有新文件且索引完好 -> 什么都不做  
    return  
  
elif not new_files and not index_exists:  
    # 索引丢失 -> 从缓存(parsed/)重建索引，不重新解析 PDF  
    ...  
else:  
    # 有新文件 -> 解析新 PDF + 重建索引  
    ...
```

Step1：扫描与增量检测

Thinking：增量处理的好处是什么？

增量处理避免了重复解析 PDF 的开销。

即使 FAISS 索引被删除，也能从 data/parsed/ 的缓存文件 直接重建索引（不需要重新调用 PDF 解析，直接调用 Embedding API即可）

Step2: 文件名解析

Step2: 文件名解析

在打开 PDF 之前，可以从文件名中提取尽可能多的元数据。 支持三种常见的研报/财报命名格式：

文件名格式	示例	可提取信息
财报_{股票}: {标题}.pdf	财报_贵州茅台: 贵州茅台2025年第三季度报告.pdf	股票名 + 标题 + 来源(官方财报)
【{券商}】{标题}.pdf	【华泰证券】中芯国际(688981): 上调目标价.pdf	券商来源 + 标题 + 股票名(从标题中)
{通用标题}.pdf	中芯国际机构调研纪要.pdf	从标题中尝试匹配股票名

```
def parse_filename(filename):
    name = os.path.splitext(filename)[0]
    # 格式1: 财报_{股票}: {标题}
    m = re.match(r"财报[_](.+?)[: ](.+)", name)
    if m:
        return {"stock_name": m.group(1), "title": m.group(2), "source": "官方财报"}
```

```
# 格式2: 【{券商}】{标题}
m = re.match(r"【(.+?)】(.+)", name)
if m:
    result = {"source": m.group(1), "title": m.group(2)}
    _detect_stock_from_title(result) # 从标题中识别股票名
    _detect_report_type(result)      # 识别报告类型(年报/季报/研报)
    return result

# 格式3: 通用 -- 尝试从标题匹配已知股票
...

```

Step2: 文件名解析

- 报告类型识别

使用正则模式匹配标题中的关键词

```
REPORT_TYPE_PATTERNS = [  
    (r"年度报告|年报", "年报"),  
    (r"半年度报告|半年报", "半年报"),  
    (r"第三季度报告|三季报", "三季报"),  
    (r"业绩点评|业绩快报", "业绩点评"),  
    (r"深度研究|研究报告", "深度研报"),  
    (r"调研纪要", "调研纪要"),  
]
```

Thinking: 是否有更灵活的方式进行解析?

Step3: PDF 文本提取

Step3: PDF 文本提取

使用 PyPDF2 逐页提取文本。关键是保留逐页信息（每页的字符数和偏移量），后续切分 chunk 时才能回溯到具体页码。

```
from PyPDF2 import PdfReader

def extract_pdf_text(pdf_path):
    reader = PdfReader(pdf_path)
    full_text = ""
    pages = []

    for page_num, page in enumerate(reader.pages, start=1):
        text = page.extract_text() or ""
        pages.append({
            "page": page_num,
            "text": text,
            "char_count": len(text), # 记录每页字符数，用于后续页码定位
        })
```

```
full_text += text
return full_text, pages    # 返回: 完整文本 + 逐页信息
```

Thinking: 为什么要逐页记录 char_count?

在阶段 5 切分 chunk 时，需要根据 chunk 在全文中的字符偏移量反推它来自哪一页。每页的 char_count 累加后构成页码边界表。

Step4：从 PDF 正文提取元数据

Step4：从 PDF 正文提取元数据

文件名能提供的信息有限，更精确的元数据需要从 PDF 正文中用正则提取 => 提取 4 类关键信息：

元数据	提取策略	搜索范围
股票代码	优先使用名称映射表，其次正则匹配 证券代码：688981	前 15000 字符
股票名称	遍历 STOCK_CODE_MAP 的 key，检查是否出现在正文中	前 8000 字符
发布日期	优先匹配带关键词的日期（"披露日期"），其次取封面日期	前 15000 字符
报告期间	匹配 "2025年第一季度"、"2024年度" 等模式	前 8000 字符

Thinking：是否有更灵活的方式进行解析？

Step4: 从 PDF 正文提取元数据

- 股票代码提取

```
def extract_stock_code(text, stock_name=""):
    # 优先: 已知映射
    if stock_name in STOCK_CODE_MAP:
        return STOCK_CODE_MAP[stock_name]

    # 次选: 从正文前 15000 字符中正则匹配
    patterns = [
        r"(?:股票代码|证券代码)[:\s]*(\d{6})", # "证券代码: 688981"
        r"[(\d{6})]", # "中芯国际(688981)"
    ]
    for pattern in patterns:
        m = re.search(pattern, text[:15000])
        if m: return m.group(1)
```

Step4: 从 PDF 正文提取元数据

- 发布日期提取（双层策略）

```
def extract_publish_date(text):
    search_text = text[:15000]

    # 第一层: 带关键词的日期（精确匹配）

    context_patterns = [
        r"(?:披露日期|公告日期|报告日期)[：：\s]*(\d{4})\s*年\s*(\d{1,2})\s*"
        r"月\s*(\d{1,2})\s*日",
    ]
    for pattern in context_patterns:
        m = re.search(pattern, search_text)
        if m:
            return f"{m.group(1)}-{m.group(2):>02}-{m.group(3):>02}"
```

```
# 第二层: 取封面前 3000 字符中的最后一个合理日期

all_dates = []
for m in re.finditer(r"(\d{4})\s*年\s*(\d{1,2})\s*月\s*(\d{1,2})\s*日",
search_text[:3000]):
    # 只取 2020~2030 年之间的日期
    if 2020 <= int(m.group(1)) <= 2030:
        all_dates.append(...)
return all_dates[-1] if all_dates else ""
```

Thinking: 为什么取最后一个日期?

PDF 封面页通常格式为：标题 -> 券商名 -> 日期。
日期在封面中位置靠后，所以取前 3000 字符中最后出现的合理日期，比取第一个更准确。 preprocess.py

Step5: 文本切分 + 元数据注入

Step5: 文本切分 + 元数据注入

这是整个流程的核心步骤。

使用 LangChain 的 RecursiveCharacterTextSplitter 将长文本切分为 800 字/片、150 字重叠的 chunk，然后为每个 chunk 注入完整的文档级元数据和页码信息。

全文文本 -> RecursiveCharacterTextSplitter -> 计算页码边界 -> 注入元数据

Step5: 文本切分 + 元数据注入

```
def chunk_with_metadata(full_text, pages, doc_metadata,
                        chunk_size=800, chunk_overlap=150):
```

1. 切分文本

```
splitter = RecursiveCharacterTextSplitter(
    separators=["\n\n", "\n", "。", ".", " ", ""],
    chunk_size=800,
    chunk_overlap=150,
)
```

```
chunks = splitter.split_text(full_text)
```

2. 构建页码边界表 (字符偏移量 -> 页码)

例: 第1页 0~2000, 第2页 2001~5000, ...

```
page_boundaries = []
```

```
offset = 0
```

```
for p in pages:
```

```
    page_boundaries.append({
```

```
        "page": p["page"],
```

```
        "start": offset,
        "end": offset + p["char_count"],
    })
    offset += p["char_count"]
```

3. 为每个 chunk 分配页码 + 注入元数据

```
result = []
```

```
for idx, chunk_text in enumerate(chunks):
```

```
    # 找到 chunk 在全文中的起始位置
```

```
    chunk_start = full_text.find(chunk_text[:100])
```

```
    # 对照页码边界表, 确定属于哪一页
```

```
    page_num = find_page(chunk_start, page_boundaries)
```

```
    result.append({
```

```
        "text": chunk_text,
```

```
        "metadata": {
```

```
            "filename": doc_metadata["filename"],
```

```
            "stock_code": doc_metadata["stock_code"],
```


Step5: 文本切分 + 元数据注入

```
"stock_name": doc_metadata["stock_name"],
"report_type": doc_metadata["report_type"],
"publish_date": doc_metadata["publish_date"],
"page":      page_num,      # 所在页码
"chunk_index": idx,        # 切片序号
}
})
return result
```

切分参数含义:

参数	值	含义
chunk_size	800	每个 chunk 最大 800 个字符 (约 400 个中文字)
chunk_overlap	150	相邻 chunk 重叠 150 字符, 避免语义被截断
separators	\n\n, \n, 。 , ., 空格	按优先级选择切分点: 先在段落处切, 再在句子处切

Thinking: 为什么每个 chunk 都要携带完整元数据?

查询时用户可能说"茅台的营收", Agent 需要按 stock_code=600519 过滤 chunk。

如果 chunk 没有 stock_code, 就无法过滤, 会混入其他股票的内容。

同样, page 字段让 Agent 能告诉用户: 数据来自年报第 42 页。

Step6: SQLite 元数据存储

Step6: SQLite 元数据存储

每个处理过的文档在 SQLite 中保存一条记录，记录文档级别的元数据。

=> 使得 Agent 可以快速查询结构化问题，比如：

有哪些文档

某只股票有几份报告

```
CREATE TABLE IF NOT EXISTS documents (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  filename    TEXT NOT NULL UNIQUE,  -- PDF 文件名（去重键）  
  title       TEXT,                  -- 报告标题  
  stock_name  TEXT,                  -- 股票名称  
  stock_code  TEXT,                  -- 股票代码 (如 688981)  
  source      TEXT,                  -- 来源 (华泰证券/官方财报/...)  
  report_type TEXT,                  -- 报告类型 (年报/季报/深度研报)  
  report_period TEXT,                -- 报告期间 (2025年度/2025年第一季度)  
  publish_date TEXT,                 -- 发布日期 (YYYY-MM-DD)  
  total_pages INTEGER,               -- PDF 总页数  
  total_chars INTEGER,               -- 提取到的总字符数  
  total_chunks INTEGER,              -- 切分后的 chunk 数量  
  created_at  TEXT DEFAULT (datetime('now'))  
);
```

Step6: SQLite 元数据存储

实际运行后的数据示例：

ID	股票	代码	类型	发布日期	页数	Chunks
1	中芯国际	688981	深度研报	2025-04-17	29	49
8	中芯国际	688981	年报	2024-06-28	222	349
10	中芯国际	688981	年报	-	232	319
11	贵州茅台	600519	三季度报	-	12	17

Step7：构建统一 FAISS 向量索引

Step7：构建统一 FAISS 向量索引

整个预处理中计算量最大、最核心的步骤。

将所有 chunk 的文本通过 DashScope 的 text-embedding-v4 模型转换为高维向量（1024维），存入 FAISS（Facebook AI Similarity Search）索引。

Thinking：FAISS 的作用是什么？

当用户提问"茅台的毛利率是多少"时，Agent 会把这个问题也转成向量，然后在 FAISS 中计算与所有 chunk 向量的余弦相似度，返回语义最相近的 chunk。

=> 理解语义含义。

Step7: 构建统一 FAISS 向量索引

```
def build_unified_index(all_chunks, index_dir, embedding_model):
    embeddings = DashScopeEmbeddings(model="text-embedding-v4")
    texts = [c["text"] for c in all_chunks]
    metadatas = [c["metadata"] for c in all_chunks]

    # 分批调用 Embedding API (每批最多 25 条)
    batch_size = 25
    vectorstore = None

    for i in range(0, len(texts), batch_size):
        batch_texts = texts[i : i + batch_size]
        batch metas = metadatas[i : i + batch_size]
        # 带重试的 API 调用 (最多 6 次, 递增等待)
        for attempt in range(6):
            try:
                if vectorstore is None:
                    vectorstore = FAISS.from_texts(batch_texts, embeddings,
metadatas=batch metas)
```

```
            else:
                vectorstore.add_texts(batch_texts, metadatas=batch metas)
            break
        except Exception:
            time.sleep((attempt + 1) * 5) # 5s, 10s, 15s, 20s, 25s, 30s

    # 保存索引 (Windows 中文路径兼容方案)
    # FAISS C++ 层 fopen 不支持中文路径, 先存临时目录再移动
    tmp_dir = tempfile.mkdtemp(prefix="faiss_")
    vectorstore.save_local(tmp_dir)
    for f in os.listdir(tmp_dir):
        shutil.move(os.path.join(tmp_dir, f), os.path.join(index_dir, f))

    # 同时保存 chunks.json (供 BM25 关键词检索使用)
    with open("chunks.json", "w") as f:
        json.dump([{"text": ..., "metadata": ...}], f)
```

Step7：构建统一 FAISS 向量索引

关键设计决策：

决策	方案	原因
分批大小	25 条/批	DashScope text-embedding-v4 的 API 限制为每次最多 25 条
重试机制	最多 6 次，递增等待	应对 DashScope API 的偶发 ConnectionError
临时目录保存	先存 ASCII 路径再移动	FAISS 的 C++ fopen 无法处理 Windows 中文路径
同时保存 chunks.json	明文 JSON 存储所有 chunk	供 BM25 关键词检索使用（RAG 混合检索需要）

性能参考：860 个 chunks 需要调用 35 批 Embedding API，实际运行耗时约 2~3 分钟（含 API 网络延迟和批间等待）。如果索引被删除但缓存存在，可从 data/parsed/ 重建索引，省去 PDF 解析时间。

CASE：研报数据预处理

预处理完成后，产出 4 类文件，共同构成 Agent 的知识库基础设施：

文件	格式	内容	被谁使用
index.faiss	FAISS 二进制	所有 chunk 的向量索引	query_report.py 向量检索
index.pkl	Pickle	FAISS docstore（chunk 元数据）	query_report.py 元数据过滤
chunks.json	JSON	860 条 chunk 文本 + 元数据	query_report.py BM25 关键词检索
documents.db	SQLite	11 条文档级元数据记录	Agent 查询文档清单、过滤条件

Agent 查询时的数据流：

用户问题 -> --stock 过滤 -> FAISS 向量检索 + BM25 关键词检索 -> RRF 融合排序 -> LLM 生成回答

检索架构

当前方案是统一索引 + chunk 级元数据过滤：

- 所有股票的 chunks 放在同一个 FAISS 索引中，
- 查询时通过 stock_code 过滤

另一方法：

按股票代码或行业建立独立的 FAISS 索引，比如如 vector_store/688981/

索引更小，检索更快

检索架构

原始 PDF 的存放优化：

当前所有 PDF 平铺在 data/reports/ 中。随着数据增长，可以按股票/行业分目录存放：

data/reports/

688981_中芯国际/ # 按"代码_名称"命名，兼顾可读性和唯一性

【上海证券】深度研究报告.pdf

【华泰证券】上调目标价.pdf

2024年年度报告.pdf

2025年年度报告.pdf

600519_贵州茅台/

2025年第三季度报告.pdf

000858_五粮液/

...

单文档处理流程

process_single_pdf() 函数串联了阶段 2~6，完整处理一个 PDF 文件：

```
def process_single_pdf(pdf_path, conn):
    filename = os.path.basename(pdf_path)
    # Step 1: 文件名解析 (阶段 2)
    file_meta = parse_filename(filename)
    # -> {stock_name: "中芯国际", source: "上海证券", title: "...", ...}

    # Step 2: PDF 文本提取 (阶段 3)
    full_text, pages = extract_pdf_text(pdf_path)
    # -> "公司名称：中芯国际集成电路...", [{page:1, char_count:2000}, ...]

    # Step 3: 内容元数据提取 (阶段 4)
    stock_code = extract_stock_code(full_text, file_meta["stock_name"])
    publish_date = extract_publish_date(full_text)
    report_period = extract_report_period(full_text)
    # -> "688981", "2025-04-17", "2025年第一季度"
```

```
# Step 4: 存入 SQLite (阶段 6)
doc_id = insert_document(conn, {
    "filename": filename, "stock_code": stock_code,
    "publish_date": publish_date, ...
})

# Step 5: 文本切分 + 元数据注入 (阶段 5)
chunks = chunk_with_metadata(full_text, pages, doc_metadata)
# -> [{text: "...", metadata: {stock_code:"688981", page:1, ...}}, ...]

# Step 6: 保存纯文本缓存 (用于增量重建)
save(parsed_dir / f"{base}_full.txt", full_text)
save(parsed_dir / f"{base}_pages.json", pages)
return doc_id, chunks
```

main() 总控逻辑

```
def main():  
    # ---- 初始化 ----  
    conn = init_db("data/documents.db")  
    pdf_files = scan_pdfs("data/reports/")  
    processed = get_processed_files(conn)  
    new_files = [f for f in pdf_files if f not in processed]  
  
    # ---- 阶段 1: 解析新 PDF ----  
    all_chunks = []  
    for filename in new_files:  
        doc_id, chunks = process_single_pdf(pdf_path, conn)  
        all_chunks.extend(chunks)
```

```
# ---- 阶段 2: 加载已处理文档的 chunks (增量重建) ----  
for filename in skip_files:  
    # 从 parsed/*_full.txt 读取缓存文本  
    # 从 SQLite 获取元数据  
    # 重新切分为 chunks (不调用 Embedding API)  
    chunks = chunk_with_metadata(cached_text, pages, db_metadata)  
    all_chunks.extend(chunks)  
  
# ---- 阶段 3: 构建 FAISS 索引 ----  
build_unified_index(all_chunks, "data/vector_store/")  
# 输出: index.faiss + index.pkl + chunks.json + index_meta.json  
  
# ---- 输出摘要 ----  
list_documents(conn)  
# 打印处理结果表格
```

CASE：研报数据预处理

运行结果：

=====

数据预处理 - PDF 批量处理 & 统一索引构建

=====

[扫描] 发现 11 个 PDF 文件

[待处理] 11 个新文件

=====

[处理] 【上海证券】中芯国际深度研究报告：晶圆制造龙头，领航国产芯片新征程.pdf

标题: 中芯国际深度研究报告：晶圆制造龙头，领航国产芯片新征程

股票: 中芯国际 | 来源: 上海证券 | 类型: 深度研报

页数: 29 | 字符: 31925 | 代码: 688981 | 发布: 2025-04-17

Chunks: 49

[处理] 【财报】中芯国际：中芯国际2024年年度报告.pdf

标题: 中芯国际2024年年度报告

股票: 中芯国际 | 来源: 官方财报 | 类型: 年报

页数: 222 | 字符: 227130 | 代码: 688981 | 发布: 2024-06-28

Chunks: 349

... (共处理 11 个文件)

[统计] 共处理 11 个新文件, 总计 860 个 chunks

[构建索引] 860 个 chunks -> data/vector_store/

[索引] 使用 text-embedding-v4 为 860 个 chunks 生成向量...

[25/860] ... [860/860] 已完成

[索引] 统一索引已保存: data/vector_store/

860 个 chunks, 11 个文档, 2 只股票 ['600519', '688981']

DeepAgents框架

DeepAgents 框架

DeepAgents:

LangChain 官方推出的 Agent 框架，构建在 LangGraph 之上。

=> 提供了一套开箱即用的 Agent 基础设施，让开发者专注于业务逻辑而非底层管道。

DeepAgents = LangGraph Agent + 内置工具 + Skills 系统 + 子代理 + 中间件栈。

开发者只需定义 `system_prompt`、`tools`、`skills`，即可获得一个具备文件操作、Shell 执行、任务规划等能力的完整 Agent。

内置工具

create_deep_agent() 创建的 Agent 默认包含以下内置工具，无需额外注册：

工具名	功能	说明
write_todos	管理任务列表	规划和追踪多步骤任务
ls	列出目录内容	文件系统探索
read_file	读取文件	查看代码、配置、SKILL.md
write_file	写入文件	创建新文件
edit_file	编辑文件	修改已有文件
glob	文件模式匹配	搜索文件路径
grep	内容搜索	在文件中搜索关键词
execute	执行 Shell 命令	运行脚本、安装包等
task	调用子代理	将任务委托给专门的子代理

DeepAgents使用

create_deep_agent 核心参数

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model=llm,          # LLM 模型（必需）
    system_prompt=prompt, # 系统提示（会拼接到框架默认提示之前）
    tools=[tool1, tool2], # 自定义工具（与内置工具合并）
    backend=backend,     # 后端（文件系统/沙箱/状态）
    skills=["/skills/"], # Skills 目录路径列表（可选）
    memory=["./AGENTS.md"], # 记忆文件路径列表（可选）
    subagents=[...],     # 子代理定义列表（可选）
    checkpointer=saver,  # 状态持久化（多轮对话）
)
```


DeepAgents使用

后端 (Backend) 选择:

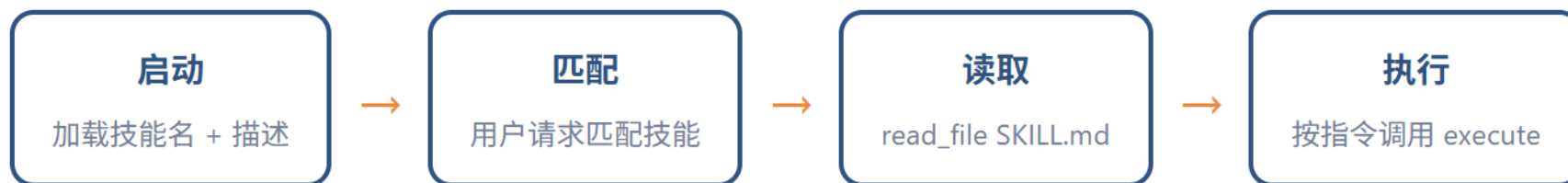
后端类型	适用场景	特点
LocalShellBackend	本地开发、脚本执行	真实的文件系统 + Shell 执行; 支持 <code>virtual_mode</code> (虚拟根目录) 和 <code>inherit_env</code> (继承环境变量)
FilesystemBackend	纯文件操作 (无 Shell)	只提供文件读写, 不支持 <code>execute</code>
StateBackend	内存中/无持久化	所有文件存在内存中, 适合测试和 LangGraph Cloud 部署

Skills 系统

Skills:

在DeepAgents中也有Skills，它是渐进式能力披露（Progressive Disclosure）机制。

Agent 启动时只看到技能名和描述，需要时再读取完整指令。



Skills 系统

Skills 目录结构

```
skills/  
  web-search/  
    SKILL.md ← 必需: YAML 前置元数据 + Markdown 指令  
    scripts/  
      search_market.py ← 可执行脚本  
  read-pdf/  
    SKILL.md  
    scripts/  
      query_report.py  
  financial-analysis/  
    SKILL.md  
    scripts/  
      ratio_analysis.py
```

name 命名规范 (Agent Skills Specification)

- 1-64 个字符
- 仅允许小写字母、数字和连字符 (a-z, 0-9, -)
- 不能以 - 开头或结尾, 不能连续 --
- 必须和包含 SKILL.md 的目录名一致

Skills 系统

SKILL.md 规范:

每个 SKILL.md 必须以 YAML 前置元数据 (frontmatter) 开头:

```
# SKILL.md 标准格式
---
name: web-search      # 必需，必须和目录名一致
description: "联网搜索..." # 必需，最多 1024 字符
keywords: 搜索, 新闻, 实时 # 可选，帮助匹配
license: MIT          # 可选
compatibility: Python 3.10+ # 可选
---

# web-search 技能指南

## 适用场景
- 查询最新股价、行情
- 获取行业政策、市场动态
...
```

```
## 可用脚本
| 脚本 | 功能 | 参数 |
|-----|-----|-----|
| `scripts/search_market.py` | 搜索 | `--query`, `--type` |

## 工作流程
1. 判断是否需要实时数据
2. 执行脚本
3. 整合结果

## 示例对话
用户: "贵州茅台最新股价？"
步骤:
1. 执行 `python skills/web-search/scripts/search_market.py --query "..." --type stock`
2. 返回结果
```

SkillsMiddleware 工作原理

当create_deep_agent(skills=["/skills/"]) 时，框架内部做了这些事：

- 扫描目录：遍历 /skills/ 下所有子目录，查找 SKILL.md
- 解析元数据：提取 YAML frontmatter 中的 name、description
- 注入 system prompt：将技能列表追加到 system prompt 末尾

注入到 system prompt 的内容格式：

```
## Skills System
```

```
You have access to a skills library...
```

```
**Available Skills:**
```

```
- **web-search**: 联网搜索实时市场信息...
```

```
-> Read `/skills/web-search/SKILL.md` for full instructions
```

```
- **read-pdf**: 读取上市公司PDF财报...
```

```
-> Read `/skills/read-pdf/SKILL.md` for full instructions
```

```
**How to Use Skills (Progressive Disclosure):**
```

1. Recognize when a skill applies
2. Read the skill's full instructions
3. Follow the skill's instructions
4. Access supporting files

SkillsMiddleware 工作原理

Qwen 兼容性问题:

SkillsMiddleware 的注入方式是为 Claude 设计的。

=> Claude 理解 progressive disclosure 模式

看到技能名后会先 read_file SKILL.md, 再用 execute 运行脚本。

但 Qwen 模型会把技能名 (如 web-search) 直接当作工具名来调用

=> 导致 "web-search is not a valid tool" 错误。

两种架构模式对比

模式一：Skills + execute (Claude 原生模式)

这是 DeepAgents 为 Claude 设计的标准模式，Claude 可以正确理解渐进式披露。

```
# Claude 原生模式
agent = create_deep_agent(
    model=ChatAnthropic(model="claude-sonnet-4-6"),
    backend=backend,
    skills=["/skills/"], # SkillsMiddleware 注入技能列表
)

# Claude 的调用流程：
# 1. 用户: "搜索贵州茅台最新业绩"
# 2. Claude: read_file("/skills/web-search/SKILL.md")
# 3. Claude: execute("python scripts/search_market.py --query ...")
# 4. Claude: 整理结果返回用户
```

使用 Claude 模型时，Skills 模式最为自然。

官方 text-to-sql-agent、content-builder-agent 示例都使用此模式。

两种架构模式对比

模式二：@tool 直接注册（Qwen 适配模式）

当使用非 Claude 模型（如 Qwen、GPT）时，参考官方 content-builder-agent 中 web_search 的定义方式，将核心能力定义为 @tool 函数。

```
# Qwen 适配模式（参考官方 content-builder-agent）
from langchain_core.tools import tool

@tool
def web_search(query: str, type: str = "general") -> str:
    """搜索网络获取最新市场信息。"""
    return _run_script([
        sys.executable, "skills/web-search/scripts/search_market.py",
        "--query", query, "--type", type,
    ])

@tool
def query_pdf(query: str, stock: str = "") -> str:
    """从本地 PDF 知识库检索（RAG）。"""
    ...
```

```
agent = create_deep_agent(
    model=ChatTongyi(model="qwen-plus"),
    tools=[web_search, query_pdf, ...], # 直接注册为工具
    backend=backend,
    # 不使用 skills=[...]
)

# Qwen 的调用流程：
# 1. 用户: "搜索贵州茅台最新业绩"
# 2. Qwen: web_search(query="贵州茅台最新业绩", type="stock")
# 3. 工具内部: subprocess 调用 search_market.py
# 4. Qwen: 整理结果返回用户
```


两种架构模式对比

维度	Skills + execute	@tool 直接注册
适合模型	Claude	Qwen / GPT / 任意模型
能力注册	SkillsMiddleware 注入 system prompt	<code>tools=[...]</code> 注册为真实工具
调用链	<code>read_file SKILL.md</code> → <code>execute</code> 脚本	直接调用 @tool 函数
错误风险	Claude 无问题；Qwen 会误调技能名	所有模型均能正确调用
灵活性	添加技能只需创建 SKILL.md 目录	需在代码中定义 @tool 函数
官方示例	text-to-sql-agent	content-builder-agent 的 <code>web_search</code>

在 投研 Agent 中，如果使用 Qwen-plus 模型 => 选择 @tool 模式。

核心能力（`web_search`、`query_pdf`、`stock_price` 等）定义为 @tool， workflow策略（五步法研报）写在 `system_prompt` 中。

=> 符合官方 content-builder-agent 的设计模式。

自定义 Skill

Thinking: 如何创建一个 Skill?

Step 1: 创建目录结构

在项目 skills/ 目录下创建

```
mkdir skills/web-search
```

```
mkdir skills/web-search/scripts
```

Step 2: 编写核心脚本

```
# skills/web-search/scripts/search_market.py
import argparse
from langchain_community.chat_models.tongyi import ChatTongyi

def search(query: str, search_type: str):
    """调用 LLM 的联网搜索能力"""
```

```
llm = ChatTongyi(
    model="qwen-plus",
    model_kwargs={"enable_search": True}
)
# 根据 type 构造不同的 prompt
prompt = f"请搜索并分析: {query}"
result = llm.invoke(prompt)
print(result.content)

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--query", required=True)
    parser.add_argument("--type", default="general",
                        choices=["general", "stock", "news", "policy"])
    args = parser.parse_args()
    search(args.query, args.type)
```

自定义 Skill

Step 3: 编写 SKILL.md

```
---  
name: web-search  
description: "联网搜索实时市场信息，获取最新股价、行业动态、政策新闻。"  
keywords: 搜索, 新闻, 实时, 行情  
---
```

web-search 技能指南

适用场景

- 查询某只股票的最新行情
- 获取最新的行业政策
- 搜索分析师评级

可用脚本

| 脚本 | 功能 | 参数 |

|-----|-----|-----|

| `scripts/search\_market.py` | 联网搜索 | `--query`, `--type` |

示例

用户: "贵州茅台今天股价多少?"

执行: `python skills/web-search/scripts/search\_market.py --query "贵州茅台最新股价" --type stock`

自定义 Skill

Step 4: 注册到 Agent

```
# Qwen 模式: 需要在代码中定义 @tool 包装
@tool
def web_search(query: str, type: str = "general") -> str:
    """搜索网络获取最新市场信息。

    Args:
        query: 搜索关键词
        type: 搜索类型 - general/news/stock/policy
    """
    return _run_script([
        sys.executable,
        "skills/web-search/scripts/search_market.py",
        "--query", query, "--type", type,
    ])
```

```
# 添加到 TOOLS 列表
TOOLS = [web_search, ...]
```

Claude 模式: 只需在 `create_deep_agent` 中设置 `skills=["/skills/"]`, 框架自动发现并加载。无需修改任何代码。

自定义 Skill

投研 Agent 的 6 个工具

工具函数	对应脚本	核心参数	功能
web_search	search_market.py	query, type	联网搜索市场信息
query_pdf	query_report.py	query, stock	本地 PDF 知识库检索 (RAG)
stock_price	get_kline.py	code, period, count	MiniQMT 实时 K 线
financial_analysis	ratio_analysis.py	stock, years	财务指标趋势分析
compare_reports_period	cross_period.py	stock, topics	同公司跨期对比
compare_reports_company	cross_company.py	stocks, topic	跨公司横向对比

CASE: content-builder-agent (内容生成 Agent)

与 投研Agent 相似的官方示例，展示了 tools + skills 混用的正确模式：

content-builder-agent/

content_writer.py ← 主程序

AGENTS.md ← 品牌人格和写作标准（memory）

subagents.yaml ← 子代理配置（研究员）

skills/

blog-post/

SKILL.md ← workflow 指导（非能力）

social-media/

SKILL.md ← workflow 指导（非能力）

CASE: content-builder-agent (内容生成 Agent)

content_writer.py 核心结构

@tool

```
def web_search(query, max_results=5, topic="general") -> dict:
    """Search the web for current information."""
    client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])
    return client.search(query, max_results=max_results, topic=topic)
```

@tool

```
def generate_cover(prompt, slug) -> str:
    """Generate a cover image for a blog post."""
    ...
```

```
def create_content_writer():
    return create_deep_agent(
        memory=["./AGENTS.md"],      # 品牌人格
        skills=["./skills/"],         #  workflow (blog-post/social-media)
        tools=[generate_cover, ...],  # 能力 (图片生成)
        subagents=load_subagents(...), # 研究员子代理
        backend=FilesystemBackend(...),
    )
```

- web_search 和 generate_cover 是能力，定义为 @tool
- blog-post 和 social-media 是 workflow，定义为 skills
- 工具名 (web_search) 和技能名 (blog-post) 不重叠
- 子代理 (researcher) 承担辅助研究任务

CASE: deep_research (深度研究 Agent)

展示了 subagents (子代理) 的使用方式:

```
# 定义研究子代理
research_sub_agent = {
    "name": "research-agent",
    "description": "Delegate research to the sub-agent.",
    "system_prompt": RESEARCHER_INSTRUCTIONS,
    "tools": [tavily_search, think_tool], # 子代理专属工具
}

# 主代理包含研究子代理
agent = create_deep_agent(
    model=model,
    tools=[tavily_search, think_tool],
    system_prompt=INSTRUCTIONS,
    subagents=[research_sub_agent],
)
```


CASE: text-to-sql-agent (SQL 查询 Agent)

展示了外部工具集成 (SQLDatabaseToolkit) + skills 混用:

```
# 集成 LangChain SQL 工具集
toolkit = SQLDatabaseToolkit(db=db, llm=model)
sql_tools = toolkit.get_tools()

agent = create_deep_agent(
    model=model,
    skills=["./skills/"], # schema-exploration + query-writing  workflow
    tools=sql_tools,      # SQL 查询/执行能力
    backend=FilesystemBackend(root_dir=base_dir),
)
```

中间件栈与执行流程

- DeepAgents 默认中间件栈

create_deep_agent 内部自动组装以下中间件（按顺序执行）：

- 1) TodoListMiddleware: 管理任务列表 (write_todos 工具)
- 2) MemoryMiddleware: 加载 AGENTS.md 记忆文件 (如配置了 memory=)
- 3) SkillsMiddleware: 扫描 skills 目录, 注入技能列表到 system prompt (如配置了 skills=)
- 4) FilesystemMiddleware: 提供 ls/read_file/write_file/edit_file/glob/grep 工具
- 5) SubAgentMiddleware: 提供 task 工具 + 子代理管理
- 6) SummarizationMiddleware: 长对话自动摘要压缩
- 7) AnthropicPromptCachingMiddleware: Claude 提示缓存优化 (非 Claude 自动忽略)
- 8) PatchToolCallsMiddleware: 修复模型输出中的工具调用格式

中间件栈与执行流程

- System Prompt 拼接顺序

最终送到 LLM 的 system prompt 由多个部分拼接而成：

```
# 最终 system prompt = 用户 prompt + 框架 prompt + 中间件注入
```

```
你是 Charles，一位专业的 AI 投研情报官... ← 用户 system_prompt
```

```
You are a Deep Agent, an AI assistant... ← BASE_AGENT_PROMPT（框架默认）
```

```
## Skills System ← SkillsMiddleware 注入（如启用）
```

```
You have access to a skills library...
```

中间件栈与执行流程

- 完整执行流程

用户输入 (HumanMessage)

→ 中间件 before (加载 skills/memory)

→ LLM 推理 (生成文本 + tool_calls)

→ 工具执行 (内置 + 自定义工具)

→ LLM 再推理 (基于工具结果继续)

→ 输出响应 (AIMessage)

工具调用和 LLM 推理可能循环多轮，直到 Agent 认为任务完成

踩坑1：Qwen + SkillsMiddleware 的兼容性问题

- Qwen + SkillsMiddleware 的兼容性问题

问题现象：使用 `skills=["/skills"]` + Qwen-plus 时，模型反复出现 "web-search is not a valid tool" 错误。

模型尝试直接调用技能名作为工具，而非按 progressive disclosure 模式先读 SKILL.md 再用 execute。

问题根因

- SkillsMiddleware 将技能名以 `**web-search**`: 描述... 格式注入 system prompt
- Qwen 模型将这些名字理解为可直接调用的工具函数
- 框架实际注册的工具列表中没有 web-search（只有 execute、read_file 等内置工具）
- 调用失败后模型会自纠正，但浪费了 1-5 次调用

踩坑1：Qwen + SkillsMiddleware 的兼容性问题

尝试过的方案

方案	效果
在 system prompt 中加强警告"不要直接调用技能名"	无效 模型仍然尝试
移除 <code>skills=["/skills"]</code>	部分有效 但丢失了技能发现能力
用 <code>StructuredTool.from_function</code> 包装	部分有效 代码复杂，参数传递有问题
动态扫描 SKILL.md 生成 StructuredTool	部分有效 过度封装，脚本选择有误
参考官方 @tool 模式	完美解决 零错误，代码简洁

踩坑2: Windows 兼容性

- Windows 兼容性

问题	解决方案
FAISS 不支持中文路径	使用 <code>tempfile</code> 临时目录保存索引，再复制回目标路径
subprocess 中文输出乱码	设置 <code>encoding="utf-8" + errors="replace" + env["PYTHONUTF8"]="1"</code>
控制台打印 <code>UnicodeEncodeError</code>	脚本开头用 <code>io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8")</code> 包装
DashScope API 连接超时	添加重试逻辑（最多 3 次，指数退避）

CASE：撰写AI投研报告

CASE：撰写AI投研报告

TO DO：搭建AI投研报告撰写Agent

用户输入：帮我撰写贵州茅台投研报告

Agent 自主完成以下全过程：



Agent 自主规划搜索策略，进行 2 轮迭代搜索（共调用 6 个工具、8 次调用），然后按国泰君安五步法框架输出完整研报。

Agent 驱动的核心：System Prompt

Agent 驱动的核心：System Prompt 即策略

直接嵌入 Agent 的 system_prompt 中，成为 Agent 的思维操作系统：

```
system_prompt = """你是 Charles，一位专业的 AI 投研情报官。
```

```
=== 研报撰写策略 ===
```

```
核心方法论: 国泰君安"五步法"
```

```
（信息差 -> 逻辑差 -> 预期差 -> 催化剂 -> 结论+风险闭环）
```

```
--- 第一阶段: 规划 ---
```

1. 识别分析对象（个股/行业/事件）
2. 判断场景类型: 个股深度 / 季报速评 / 行业比较 / 事件驱动 / 财务异常
3. 列出每一步需要搜集什么信息
4. 规划搜索序列

system_prompt 里不写如何调用脚本，而是写如何思考。

Agent 看到五步法的思考框架后，自己决定在每一步该调用哪些工具、搜索什么内容。

```
--- 第二阶段: 迭代式信息收集 ---
```

```
以 web_search 为主要信息来源，通过多轮搜索逐步积累：
```

- 第一轮: 搜索基本面概况
- 分析结果: 发现新线索、新问题
- 第二轮: 针对线索追加搜索（关键 -- 不要一次搜完就停）
- 继续迭代...

```
--- 第三阶段: 五步法分析与输出 ---
```

```
Step 1 信息差: 3-5个被市场忽视的关键数据点
```

```
Step 2 逻辑差: 市场误读 vs 正确逻辑的对比
```

```
Step 3 预期差: 量化偏离幅度的对比表
```

```
Step 4 催化剂: 按时间排序的催化剂清单
```

```
Step 5 结论+风险闭环: 核心观点 + 失效条件"""
```

@tool 注册模式

参考 DeepAgents 的 content-builder-agent, 将核心能力定义为 @tool 函数, Agent 可以直接调用:

```
from langchain_core.tools import tool

@tool
def web_search(query: str, type: str = "general") -> str:
    """搜索网络获取最新市场信息。"""
    return _run_script([sys.executable,
                        "skills/web-search/scripts/search_market.py",
                        "--query", query, "--type", type])

@tool
def query_pdf(query: str, stock: str = "") -> str:
    """从本地 PDF 知识库检索 (RAG)。"""
    ...
```

```
@tool
def financial_analysis(stock: str, years: int = 5) -> str:
    """分析财务指标趋势 (ROE/毛利率/负债率等)。"""
    ...

# Agent 创建 -- tools= 注册为真正可调用的工具
agent = create_deep_agent(
    model=ChatTongyi(model="qwen-plus"),
    system_prompt=system_prompt, # 五步法策略
    tools=[web_search, query_pdf, # 6 个 @tool
           stock_price, financial_analysis,
           compare_reports_period, compare_reports_company],
    backend=backend,
)
```

@tool 注册模式

Thinking: 为什么用 @tool 而不是 DeepAgents 的 skills=["/skills"]?

因为 skills 机制是为 Claude 设计的 渐进式披露 模式。

Qwen 模型会把技能名（如 web-search）当成工具名来调用，导致报错。

@tool 是模型无关的标准方式，任何 LLM 都能正确调用。

五步法的信息来源策略

步骤	主要工具	搜索/查询内容	关注维度
信息差	query_pdf + web_search	财报附注、现金流、非经常性损益	被忽视的数据
逻辑差	Agent 自主推理	基于前步数据构建因果链	市场的错误推理
预期差	web_search + financial_analysis	券商一致预期、历史趋势	偏离幅度与持续性
催化剂	web_search(news/policy)	近期事件、政策、行业动态	触发重估的时间节点
结论+风险闭环	Agent 综合判断	整合前四步 + stock_price 行情验证	投资评级 + 失效条件

风险闭环的意义：不是简单列一堆风险因素，而是明确指出“如果 x 发生，前面四步的分析将全部失效”。

比如：如果中芯国际的投资逻辑建立在 国产替代加速 的假设上，那么 出口管制大幅放松导致客户回流台积电 就是让整个逻辑崩塌的失效条件。

Agent的工具调用过程

用户输入： "帮我撰写贵州茅台投研报告 "

第一轮（并行）： 同时发起 3 个工具调用

工具	调用参数	返回
query_pdf	query="贵州茅台 2025年年报", stock="600519"	匹配 17 个 chunks, RAG 问答
web_search	query="贵州茅台 2025年年报 业绩公告", type="stock"	发现 2025 年报尚未发布
financial_analysis	stock="600519", years=5	营收 1741 亿(+15.66%)、 ROE 36.02%

第二轮（迭代追加）： 发现年报未发布后， 自动调整策略

工具	调用参数	返回
web_search	query="贵州茅台 2026年一季度业绩预告", type="stock"	Q1 营收 580-590 亿
stock_price	code="600519.SH", period="1d", count=30	近 30 日 K 线, 最新价 1468 元

Agent 的自主性： 第一轮搜索发现 2025 年报尚未发布 后， 没有停下来报错， 而是自动追加搜索 2026Q1 业绩预告和实时行情。 => 这就是 迭代式信息收集的价值： 发现线索， 追加搜索。

工具调用汇总

工具	调用次数	参数示例	返回内容
web_search	3 次	年报状态、年报全文、Q1业绩预告	最新市场信息、业绩数据
query_pdf	1 次	stock=600519, 关键财务数据	本地三季报 RAG 检索结果
financial_analysis	1 次	stock=600519, years=5	5 年核心指标趋势分析
stock_price	1 次	600519.SH, 1d, 30	近 30 日 K 线及最新价

全部工具调用均成功，无任何 "not a valid tool" 错误。

=> 将核心能力定义为 @tool 函数（参考 DeepAgents 官方 content-builder 模式），而非依赖 SkillsMiddleware 的技能名注入。

Agent 输出的完整研报

贵州茅台（600519.SH）深度投研报告

高壁垒下的确定性增长与估值再锚定

日期：2026年4月3日 | 分析师：Charles, AI 投研情报官

执行摘要

贵州茅台作为中国白酒行业的绝对龙头与全球稀缺消费资产，其核心价值在于无与伦比的品牌护城河、极致的现金流生成能力及持续优化的渠道结构。

- 短期（1-3 个月）：**2026 年一季报（预计 4 月 15-20 日披露）将成为关键催化剂，预示营收/净利双位数增长
- 中期（6-12 个月）：**"i茅台"直营体系的成熟与系列酒改革深化，将系统性提升盈利质量
- 长期：**增长逻辑从"量价齐升"转向"结构优化+效率提升"，ROE 中枢稳定在 35% 以上
- 评级：** 增持 目标价区间 1520-1680 元（2026E PE 28-32x）

Step 1 信息差 -- 市场还不知道/忽视了什么？

市场普遍关注茅台的"价格"与"销量"，但以下财报细节被显著低估：

- "i茅台"财务贡献远超认知：**2026Q1 直销收入预计达 280 亿元，占当季总营收 48%-49%。单瓶直销毛利较经销模式高约 330 元，直接拉动净利率跃升至 51%-53%。市场仍将其视为"渠道补充"。
- 系列酒"去库存化"已实质性完成：**2025 年三季报显示库存天数降至 45 天以内，为近五年最低。2026 年有望贡献超 120 亿元净利润（2024 年为 89 亿元）。
- 现金流与利润背离正在收窄：**2024 年经营性现金流净额 924.64 亿元（+38.85%），远超归母净利润 862.28 亿元（+15.38%）。代表更真实、更可持续的盈利质量。

Agent 输出的完整研报

Step 2 逻辑差 -- 市场的推理错在哪里？

市场普遍认知	正确逻辑链	数据支撑
"高端白酒已见顶，增长乏力"	增长逻辑已切换为"集中度提升"，茅台凭品牌力持续夺取份额，高端市场份额从 2020 年 45% 提升至 2025 年 58%	2024 年营收 +15.66%，2026Q1 预告 +12.7%-14.7%，均远超行业平均 5%-7%
"直销冲击批价，损害渠道信心"	"i茅台"构建了三层渠道防火墙。2026 年春节后飞天批价已回升至 1650-1700 元	经销商库存仅 30-45 天，i茅台订单满足率稳定在 65%
"高估值不可持续，必须等回调"	应锚定"永续现金流折现"属性。91.93% 毛利率、36.02% ROE、19.04% 负债率，A 股独一无二	近 5 年 ROE 均值 33.5%，波动率仅 2.1%

Step 3 预期差 -- 一致预期 vs 我的预测

指标	券商一致预期 (2026E)	Charles 预测 (2026E)	偏离幅度
营业总收入	1,920 亿元	1,955 亿元	+1.8%
归母净利润	975 亿元	1,012 亿元	+3.8%
基本 EPS	77.5 元	80.4 元	+3.8%
ROE（加权）	35.2%	36.1%	+0.9pct
目标价（PE 法）	1,480 元 (30x)	1,600 元 (31x)	+8.1%

市场预期总体审慎，但低估了直营化与产品结构优化对盈利质量的提升幅度。

Agent 输出的完整研报

Step 4 催化剂 -- 引爆重估的时间轴

时间	催化剂	影响	风险
2026 年 4 月 15-20 日	一季报正式发布	若营收/净利落在预告上限，将打消市场疑虑	数据落在预告下限，管理层展望不及预期
2026 年 4 月下旬	2025 年年报发布	验证"高质量发展"战略落地的关键证据	非经常性损益或会计政策变更
2026 年 5-6 月	端午节动销数据	巩固全年增长信心	宏观经济走弱影响消费情绪
2026 年 Q3 末	2027 年生肖酒发布	"价格锚"与"情绪放大器"	发行量过大或定价过低

Step 5 结论 + 风险闭环

投资建议：维持"增持"评级。当前股价 1468 元，对应 2026E PE 约 29.5 倍，处于历史估值中枢偏下，具备安全边际与上行空间。

失效条件（风险闭环）-- 哪个假设出错会导致结论崩塌：

- "i茅台"可持续性：**若监管政策变化导致 i 茅台增长失速，直营化带来的利润增厚将不复存在
- 高端白酒消费刚性：**若经济深度衰退导致高端消费场景系统性坍塌（频次下降 30%+），增长逻辑将动摇
- 公司治理与战略定力：**若管理层偏离"高质量发展"主线（如激进扩产或价格战），将损害长期品牌价值

风险提示：宏观经济不及预期风险；白酒行业政策变动风险；食品安全风险；汇率波动风险（影响海外业务）；研究报告使用的公开资料可能存在信息滞后或误差。

Agent 输出的研报要点

Agent 收集完信息后，按五步法框架直接在对话中输出完整 Markdown 研报。

- 信息差：i 茅台直销贡献远超认知（占比 48%-49%）；系列酒去库存已完成；经营现金流 924 亿（+38.85%）远超净利润增速
- 逻辑差："高端白酒见顶"是线性外推的错误 -- 增长逻辑已从行业扩容切换为集中度提升。"直销冲击批价"也是误读 -- 三层渠道防火墙有效协同
- 预期差：2026E 归母净利润预测 1012 亿 vs 一致预期 975 亿，正向偏离 +3.8%
- 催化剂：4 月中旬 Q1 报 → 4 月底年报 → 端午动销 → Q3 生肖酒发布
- 风险闭环：3 个失效条件，i 茅台政策风险、高端消费系统性坍塌、管理层偏离战略定力

Summary

五步法的本质是认知博弈

=> 不是简单的基本面分析工具，而是一套发现市场认知偏差并利用偏差获利的方法论。

- 1) 信息差：市场不知道什么？
- 2) 逻辑差：市场知道了但理解错了什么？
- 3) 预期差：市场的预期偏离了多少？
- 4) 催化剂：什么事件会迫使市场修正认知？
- 5) 风险闭环：我的分析可能错在哪？

对于 AI Agent 来说，五步法的价值在于提供了一个结构化的分析框架，让 LLM 不是泛泛而谈，而是沿着"数据 -> 推理 -> 量化 -> 事件 -> 结论"的链条递进分析。

@tool 提供数据获取能力，system_prompt 提供思维框架，LLM 提供推理能力 => 产出有价值的研报。

Summary

System Prompt 设计原则:

好的 system prompt 不是角色描述, 而是 Agent 的操作系统:

- 工具清单: 告诉 Agent 有哪些能力可用
- workflow 策略: 面对不同任务类型的处理路径
- 思考框架: 如五步法的每一步核心问题和输出格式
- 场景模板: 常见场景的推荐工具组合
- 数据说明: 本地有哪些数据、常用代码等背景知识

Summay (@tool 函数编写最佳实践)

```
@tool
```

```
def web_search(query: str, type: str = "general") -> str:
```

```
    """搜索网络获取最新市场信息。    ← 第一行：简洁描述，LLM 据此决定是否调用
```

```
    Args:
```

```
        query: 搜索关键词        ← 参数说明越清晰，LLM 传参越准确
```

```
        type: 搜索类型 - general/news/stock/policy
```

```
    """
```

```
    return _run_script([...])    ← 统一用 _run_script 处理编码和超时
```

- docstring 第一行是工具描述，LLM 据此判断何时使用该工具
- Args 越明确（含可选值枚举），LLM 传参越精准
- 返回值为纯文本字符串，方便 LLM 理解和引用
- 错误信息也返回字符串（不抛异常），让 LLM 自行决定如何处理

Summay (核心代码结构)

agent.py 核心架构 (参考官方 content-builder-agent)

1. 定义 @tool 函数 -- 能力即工具

@tool

def web_search(query: str, type: str = "general") -> str:

"""搜索网络获取最新市场信息"""

return _run_script([sys.executable,
"skills/web-search/scripts/search_market.py",
"--query", query, "--type", type])

@tool

def query_pdf(query: str, stock: str = "") -> str:

"""从本地 PDF 知识库检索 (RAG) """

...

@tool

def financial_analysis(stock: str, years: int = 5) -> str:

"""分析财务指标趋势"""

...

2. 工具列表

TOOLS = [web_search, query_pdf, stock_price,
financial_analysis, compare_reports_period,
compare_reports_company]

3. 创建 Agent -- tools= 替代 skills=

agent = create_deep_agent(

model=llm,

system_prompt=system_prompt, # 五步法策略在这里

backend=backend,

tools=TOOLS, # @tool 直接注册

)

能力用 @tool, workflow 用 system_prompt。 官方 content-builder 示例中 web_search 是 @tool, blog-post 是 skill (workflow)。两者定位不同, 不混用。

打卡：撰写AI投研报告



搭建AI投研报告撰写 Agent，完成以下任务：

- 选择一家你感兴趣的上市公司
- 先用 preprocess.py 或 read-pdf 技能读取其最新年报
- 在终端运行 python agent.py，输入"帮我撰写XX深度投研报告"
- 观察 Agent 的工具调用过程：它选择了哪些工具？搜索了几轮？
- 重点评估研报质量：

信息差：是否找到了被市场忽视的关键数据？

逻辑差：是否揭示了市场的错误推理？

风险闭环：是否明确指出了 哪个假设出错会导致结论崩塌？



Thank You
Using data to solve problems

